

# Assignment 5 : ViT-S LoRA Fine-tuning & Adversarial Attacks

---

**Student:** Nisarg Upadhyay | **Roll No:** B23CS1075 | **Branch:** Assignment\_5

---

## Links

| Resource                 | URL                                                 |
|--------------------------|-----------------------------------------------------|
| WandB (Q1 - ViT LoRA)    | <a href="#">assignment5-vit-lora</a>                |
| WandB (Q2 - Adversarial) | <a href="#">assignment5-adversarial</a>             |
| HuggingFace Model        | <a href="#">b23cs1075-assignment5-vit-lora-best</a> |
| GitHub Pages             | <a href="#">Portfolio</a>                           |

---

## Installation

```
# Clone and checkout branch
git clone https://github.com/NisargUpadhyayIITJ/ML0ps-NisargUpadhyay-B23CS1075.git
cd ML0ps-NisargUpadhyay-B23CS1075
git checkout Assignment_5

# Create virtual environment
python -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

## Docker Setup

```
docker build -t assignment5 .
docker run --gpus all -it assignment5 bash
```

---

## Q1: ViT-S LoRA Fine-tuning on CIFAR-100

### How to Run

```
# Train baseline (no LoRA) - head only
python -u q1_vit_lora/train.py --epochs 10 --lr 1e-4 --batch-size 64 --
```

only baseline

```
# Train all 9 LoRA configurations
for r in 2 4 8; do for a in 2 4 8; do
    python -u q1_vit_lora/train.py --epochs 10 --lr 1e-4 --batch-size 64 --
only r${r}_a${a}
done; done

# Test all models
python -u q1_vit_lora/test.py

# Optuna hyperparameter search
python -u q1_vit_lora/optuna_search.py --n-trials 20 --optuna-epochs 5 --
full-epochs 10
```

### Training Results — Baseline (No LoRA)

| Epoch | Training Loss | Validation Loss | Training Accuracy | Validation Accuracy |
|-------|---------------|-----------------|-------------------|---------------------|
| 1     | 2.6196        | 1.4334          | 43.40%            | 66.12%              |
| 2     | 1.0867        | 1.0085          | 73.13%            | 73.34%              |
| 3     | 0.8578        | 0.8839          | 77.23%            | 75.94%              |
| 4     | 0.7642        | 0.8244          | 78.73%            | 77.18%              |
| 5     | 0.7071        | 0.7904          | 80.15%            | 77.72%              |
| 6     | 0.6739        | 0.7694          | 80.97%            | 78.14%              |
| 7     | 0.6546        | 0.7567          | 81.33%            | 78.42%              |
| 8     | 0.6473        | 0.7501          | 81.55%            | 78.54%              |
| 9     | 0.6369        | 0.7470          | 81.84%            | 78.68%              |
| 10    | 0.6293        | 0.7464          | 81.90%            | 78.68%              |

### Test Results Summary

| LoRA | Rank | Alpha | Dropout | Overall Test Accuracy | Trainable Parameters |
|------|------|-------|---------|-----------------------|----------------------|
| No   | N/A  | N/A   | N/A     | <b>78.43%</b>         | 38,500               |
| Yes  | 2    | 2     | 0.1     | 87.52%                | 75,364               |
| Yes  | 2    | 4     | 0.1     | 88.58%                | 75,364               |
| Yes  | 2    | 8     | 0.1     | <b>88.87%</b>         | 75,364               |
| Yes  | 4    | 2     | 0.1     | 88.07%                | 112,228              |
| Yes  | 4    | 4     | 0.1     | 88.32%                | 112,228              |
| Yes  | 4    | 8     | 0.1     | 88.78%                | 112,228              |

| LoRA | Rank | Alpha | Dropout | Overall Test Accuracy | Trainable Parameters |
|------|------|-------|---------|-----------------------|----------------------|
| Yes  | 8    | 2     | 0.1     | 88.40%                | 185,956              |
| Yes  | 8    | 4     | 0.1     | 88.37%                | 185,956              |
| Yes  | 8    | 8     | 0.1     | 88.74%                | 185,956              |

### Optuna Best Configuration

| Parameter            | Value         |
|----------------------|---------------|
| Rank                 | 13            |
| Alpha                | 16            |
| Dropout              | 0.061         |
| Validation Accuracy  | 88.44%        |
| <b>Test Accuracy</b> | <b>89.35%</b> |
| Trainable Parameters | 278,116       |

### Key Observations

- **LoRA significantly improves accuracy:** All LoRA configurations (87.5–89.3%) outperform the baseline (78.4%) by ~10%.
- **Alpha matters more than rank:** Higher alpha values consistently yield better results. r2\_a8 (88.87%) with only 75K params nearly matches r8\_a8 (88.74%) with 186K params.
- **Parameter efficiency:** LoRA achieves 88%+ accuracy with just 0.35% trainable parameters.
- **Optuna found a better config:** r=13, α=16, d=0.061 achieved 89.35% — the best result.

### Training Curves & Visualizations

Training curves, class-wise accuracy histograms, and gradient norm plots are saved in [results/q1/](#) and uploaded to [WandB](#).

## Q2: Adversarial Attacks using IBM ART

### How to Run

```
# Train ResNet-18 on CIFAR-10
python -u q2_adversarial/train_resnet.py --epochs 30 --lr 0.1 --batch-size 128

# FGSM Attack comparison (scratch vs IBM ART)
python -u q2_adversarial/fgsm_attack.py --model-path weights/q2/resnet18_cifar10_best.pth

# Adversarial Detectors (PGD + BIM)
```

```
python -u q2_adversarial/detector_train.py --model-path
weights/q2/resnet18_cifar10_best.pth --detector-epochs 15 --eps 0.03
```

## Q2(i): FGSM Attack Results

**ResNet-18 Clean Test Accuracy: 93.75%**

| Epsilon ( $\epsilon$ ) | Clean Accuracy | FGSM Scratch Accuracy | FGSM ART Accuracy |
|------------------------|----------------|-----------------------|-------------------|
| 0.000                  | 93.75%         | 93.75%                | 93.75%            |
| 0.005                  | 93.75%         | 45.45%                | 49.83%            |
| 0.010                  | 93.75%         | 27.27%                | 31.65%            |
| 0.020                  | 93.75%         | 18.80%                | 22.82%            |
| 0.040                  | 93.75%         | 14.92%                | 18.11%            |
| 0.080                  | 93.75%         | 12.01%                | 13.55%            |
| 0.100                  | 93.75%         | 11.55%                | 12.39%            |
| 0.150                  | 93.75%         | 10.32%                | 10.62%            |
| 0.200                  | 93.75%         | 10.19%                | 10.27%            |
| 0.300                  | 93.75%         | 10.28%                | 10.30%            |

### Analysis:

- Even tiny perturbations ( $\epsilon=0.005$ ) reduce accuracy from 93.75% to ~47%.
- The from-scratch FGSM implementation produces slightly stronger attacks (lower victim accuracy) compared to IBM ART at the same  $\epsilon$ , likely due to minor implementation differences in gradient computation.
- At high  $\epsilon$  ( $\geq 0.2$ ), both methods converge to ~10% (random chance for 10 classes).

## Q2(ii): Adversarial Detection Results

| Attack | Detection Accuracy | Precision | Recall | F1-Score |
|--------|--------------------|-----------|--------|----------|
| PGD    | 50.85%             | 0.5909    | 0.0066 | 0.0131   |
| BIM    | <b>98.98%</b>      | 0.9803    | 0.9990 | 0.9895   |

### Analysis:

- **BIM detector** achieves excellent detection accuracy of 98.98% with near-perfect precision and recall, indicating that BIM perturbations create distinctive patterns easily identified by a ResNet-34 detector.
- **PGD detector** needs further tuning — PGD creates more subtle perturbations that are harder to detect. The adversarial examples generated by PGD are closer to the original distribution making binary classification challenging.

## Qualitative Results

Visual comparisons of clean vs adversarial images (FGSM with/without ART, PGD, BIM) are saved in [results/q2/](#) and uploaded to [WandB](#).

---

## Project Structure

```
|— Dockerfile          # Docker container setup
|— README.md          # This file
|— requirements.txt    # Python dependencies
|— q1_vit_lora/        # Q1: ViT-S LoRA experiments
|   |— model.py        # ViT-S model + LoRA injection
|   |— train.py        # Training (baseline + 9 LoRA configs)
|   |— test.py         # Testing & evaluation
|   |— optuna_search.py # Optuna hyperparameter search
|   |— utils.py        # Data loading, metrics, plotting
|— q2_adversarial/     # Q2: Adversarial attacks
|   |— model.py        # ResNet-18/34 definitions
|   |— train_resnet.py # ResNet-18 CIFAR-10 training
|   |— fgsm_attack.py  # FGSM: scratch vs IBM ART
|   |— detector_train.py # PGD/BIM adversarial detectors
|   |— utils.py        # Data loading, visualization
|— weights/           # Model weights
|   |— q1/             # ViT-S baseline + LoRA + Optuna best
|   |— q2/             # ResNet-18 + detectors
|— results/           # Plots, tables, JSON results
|   |— q1/             # Q1 results
|   |— q2/             # Q2 results
|— docs/              # GitHub Pages
|   |— index.html
|   |— assignment5.html
|   |— style.css
```